You'll notice that the `img` element can serve as an XSS vector here, using a malformed `src` attribute attack. `strip_tags()` is a bit of a blunt instrument and does not care about well-formedness or validation. Nor is this function an HTML parser, and malformed markup will likely result in more text being removed than desired. Speaking of HTML parsers, I highly recommend using one to validate and sanitize any markup injection you are permitting, rather than this function.

But Couldn't I Just Use a Regular Expression?

No. **Thou shalt not parse (X)HTML with a regex.** HTML is not a regular language. HTML is a context-free grammar. It is not possible to express the grammar of HTML using regular expressions, which is the path of madness. Do not be tempted to hunt for chevron characters and tag names; you will uncover things that are *best left undisturbed*. Follow this link for a detailed explanation: https://stackoverflow.com/a/1732454

Filter Functions

PHP – since way back in the days of 5.2 – also has an entire module devoted to finding and cleaning data: the `filter` extension will be installed by default. It comprises a handful of functions and a slew of constants to control behavior.

Table 11-42. *PHP `filter_*` functions*

`filter_input()` `filter_input_array()`	Return filtered data direct from original input (not `$_GET`, `$_POST`, etc.)
`filter_var()` `filter_var_array()`	Filter data supplied as argument
`filter_list()`	Returns array of supported filter names
`filter_has_var()`	Test if variable exists in original input data
`filter_id()`	Get numerical value of filter by name

Some of these functions are a bit quirky and interact with the input data supplied to the script. And I don't mean the `$_GET` or `$_POST` superglobals; I mean the data from which the superglobals are constructed. For example, `filter_has_var(INPUT_POST, $varName)` would check if `$varName` existed in the original POST data, not `$_POST`.